Lecture 29: Givens Rotations; Convergence of the QR Algorithm

Math 6610

C. Mou

Department of Mathematics and Statistics
Utah State University

Fall 2026

Today's Goals

- Givens rotations
- Use of Givens rotations in the QR algorithm
- Convergence of the QR algorithm
- Strategy for choosing μ_k

The QR Algorithm: QR Factorization

Definition (*QR Factorization*). *QR factorization* of $M \in \mathbb{C}^{n\times n}$:

$$M = QR, \qquad \text{where } Q \in \mathbb{C}^{n\times n} \text{ is unitary and } R \text{ is upper-triangular}$$

This is done via Gram-Schmidt.

The QR Algorithm

Remark (*QR Algorithm (for eigenvalue computations)*).

Input: upper-Hessenberg matrix $H \in \mathbb{C}^{n \times n}$

- Set $A^{(0)} := H$
- For $k = 1, 2, \ldots$
  - Choose a suitable "shift" $\mu_k \in \mathbb{C}$
  - Compute a QR factorization:

$$A^{(k-1)} - \mu_k I = Q^{(k)} R^{(k)}$$

  - Set $A^{(k)} := R^{(k)} Q^{(k)} + \mu_k I$
- end (k)

The QR Algorithm: Lemma

Lemma.

1. $A^{(k)} = (Q^{(k)})^H A^{(k-1)} Q^{(k)}, \ k = 1, 2, \ldots$
2. $\Lambda(H) = \Lambda(A^{(k)}), \ k = 0, 1, 2, \ldots$

Proof.

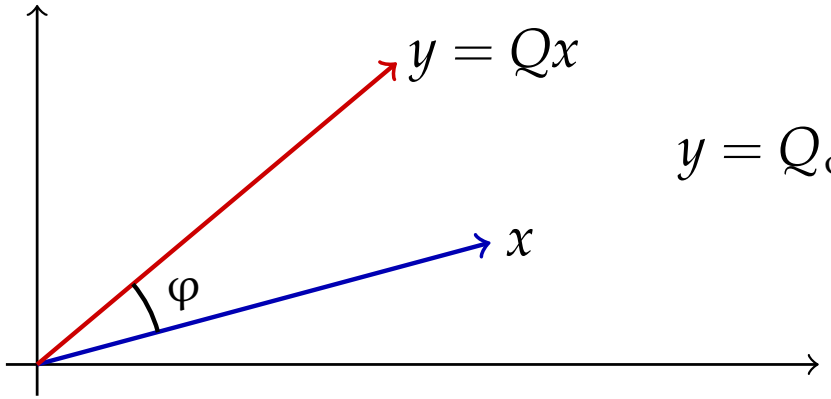
2. By (1), $A^{(k)}, A^{(k-1)}, A^{(k-2)}, \ldots, A^{(1)}, A^{(0)} = H$ are all similar.

1.
$$A^{(k)} = R^{(k)} Q^{(k)} + \mu_k I = (Q^{(k)})^H \big(\underbrace{Q^{(k)} R^{(k)} + \mu_k I}_{= A^{(k-1)}} \big) Q^{(k)}$$

■

Note: All the matrices $A^{(k)}$ are upper-Hessenberg!

Givens Rotations: 2×2 Case



$$Q = \begin{bmatrix} c & s \\ -\bar{s} & \bar{c} \end{bmatrix} \in \mathbb{C}^{2\times 2}, \qquad \text{where } |c|^2 + |s|^2 = 1$$

Q is unitary:

$$Q^H Q = \begin{bmatrix} \bar{c} & -s \\ \bar{s} & c \end{bmatrix} \begin{bmatrix} c & s \\ -\bar{s} & \bar{c} \end{bmatrix} = \begin{bmatrix} |c|^2 + |s|^2 & 0 \\ 0 & |c|^2 + |s|^2 \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

Householder vs. Givens

Householder:

$$Q \begin{bmatrix} * \\ * \\ \vdots \\ * \end{bmatrix} = \begin{bmatrix} * \\ 0 \\ \vdots \\ 0 \end{bmatrix}$$

Givens:

$$Q \begin{bmatrix} * \\ * \end{bmatrix} = \begin{bmatrix} * \\ 0 \end{bmatrix}$$

Givens Rotations: Zeroing Out x_2

Use of Q: to "zero out" the entry x_2 of any given $x = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} \in \mathbb{C}^2$, $x \neq 0$.

Indeed: Set

$$c = e^{i\alpha} \frac{\overline{x_1}}{\|x\|_2}, \qquad s = e^{i\alpha} \frac{\overline{x_2}}{\|x\|_2}, \qquad \text{where } \alpha \in \mathbb{C}$$

Then

$$|c|^2 + |s|^2 = \frac{|x_1|^2 + |x_2|^2}{\|x\|_2^2} = 1$$

$$Q \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \frac{1}{\|x\|_2} \begin{bmatrix} e^{i\alpha}\overline{x_1} & e^{i\alpha}\overline{x_2} \\ -e^{-i\alpha}x_2 & e^{-i\alpha}x_1 \end{bmatrix} \begin{bmatrix} x_2 \\ x_2 \end{bmatrix} = \begin{bmatrix} e^{i\alpha}\|x\|_2 \\ 0 \end{bmatrix}$$

Givens Rotations: General Case

$$Q = \begin{bmatrix} 1 & & & & & & & & 0 \\ & \ddots & & & & & & & \\ & & 1 & & & & & & \\ & & & c & & & s & & \\ & & & & 1 & & & & \\ & & & & & \ddots & & & \\ & & & & & & 1 & & \\ & & & -\bar{s} & & & \bar{c} & & \\ & & & & & & & 1 & \\ & & & & & & & & \ddots \\ 0 & & & & & & & & 1 \end{bmatrix}$$

Givens Rotations: General Case (Notes)

Notes:

1. Left-multiplication with Q only changes the jth and kth rows.
2. Right-multiplication with Q only changes the jth and kth columns.
3. Q is a Givens rotation $\Rightarrow Q^H$ is a Givens rotation:

$$Q^H = \begin{bmatrix} \overline{c} & -s \\ \overline{s} & c \end{bmatrix}$$

Use in QR Algorithm: Example $n = 5$ (1/3)

Example ($n = 5$).

$$A^{(k-1)} - \mu_k I = \begin{bmatrix} * & * & * & * & * \\ * & * & * & * & * \\ 0 & * & * & * & * \\ 0 & 0 & * & * & * \\ 0 & 0 & 0 & * & * \end{bmatrix}$$

Choose $Q_1 = \begin{bmatrix} c_1 & s_1 & & & \\ -\overline{s_1} & \overline{c_1} & & & \\ & & 1 & & \\ & & & 1 & \\ & & & & 1 \end{bmatrix}$ such that

$$Q_1^H (A^{(k-1)} - \mu_k^T) = \begin{bmatrix} \# & \# & \# & \# & \# \\ 0 & \# & \# & \# & \# \\ 0 & * & * & * & * \\ 0 & 0 & * & * & * \\ 0 & 0 & 0 & * & * \end{bmatrix}$$

Use in QR Algorithm: Example $n = 5$ (2/3)

Example ($n = 5$, *continued*). Choose $Q_2 = \begin{bmatrix} 1 & & & & \\ & c_1 & s_1 & & \\ & -\overline{s_1} & \overline{c_1} & & \\ & & & 1 & \\ & & & & 1 \end{bmatrix}$ such that

$$Q_2^H Q_1^H (A^{(k-1)} - \mu_k I) = \begin{bmatrix} \# & \# & \# & \# & \# \\ 0 & \# & \# & \# & \# \\ 0 & * & * & * & * \\ 0 & 0 & * & * & * \\ 0 & 0 & 0 & * & * \end{bmatrix}$$

Use in QR Algorithm: Example $n = 5$ (3/3)

Example ($n = 5$, *continued*).

$$\vdots$$

$$Q_4^H Q_3^H Q_2^H Q_1^H (A^{(k-1)} - \mu_k I) = \begin{bmatrix} * & * & * & * & * \\ 0 & * & * & * & * \\ 0 & 0 & * & * & * \\ 0 & 0 & 0 & \# & \# \\ 0 & 0 & 0 & 0 & \# \end{bmatrix} =: R^{(k)}$$

QR Factorization (Continued)

For general $n \geqslant 2$

Convert A to a Hessenberg matrix and feed this to the QR algorithm: $A \to H = A^{(0)}$.

$$A^{(k-1)} - \mu_k I = Q^{(k)} R^{(k)} \qquad \text{where } Q^{(k)} = \underbrace{Q_1 Q_2 \dots Q_{n-1}}_{n-1 \text{ Givens rotations}}$$

$$A^{(k)} = R^{(k)} Q^{(k)} + \mu_k I \qquad \text{is upper-Hessenberg}$$

Convergence of the QR Algorithm

Let $\Lambda(A) = \{\lambda_1, \lambda_2, \ldots, \lambda_n\}$ and assume that $|\lambda_1| > |\lambda_2| > \cdots > |\lambda_n|$.
(Special case: $\mu_k = 0$ for all k.) Then

$$A^{(k)} = \begin{bmatrix} * & * & \cdots & \cdots & * \\ a_{21}^{(k)} & * & \cdots & \cdots & * \\ & a_{32}^{(k)} & \ddots & & \vdots \\ & & \ddots & \ddots & \vdots \\ 0 & & & a_{n,n-1}^{(k)} & * \end{bmatrix} \xrightarrow{k\to\infty} R = \begin{bmatrix} r_{11} & * & \cdots & \cdots & * \\ 0 & r_{22} & \ddots & & \vdots \\ \vdots & \ddots & \ddots & \ddots & \vdots \\ \vdots & & \ddots & \ddots & * \\ 0 & \cdots & \cdots & 0 & r_{nn} \end{bmatrix}$$

where $\Lambda(R) = \{r_{11}, r_{22}, \ldots, r_{nn}\} = \Lambda(A)$.

Convergence of the QR Algorithm: Speed

Speed of convergence: If $|\lambda_n| > 0$, then

$$a_{j,j-1}^{(k)} = O\left(\left|\frac{\lambda_0}{\lambda_{j-1}}\right|^k\right), \qquad j = 2, 3, \ldots, \qquad k = 1, 2, \ldots$$

$\Rightarrow$ slow convergence if $|\lambda_j| \lessapprox |\lambda_{j-1}|$. Shifts μ_k are used to speed up convergence. Suppose $\mu_k = \mu =$ constant and $|\lambda_1 - \mu| > |\lambda_2 - \mu| > \cdots > |\lambda_n - \mu| > 0$. Speed of convergence:

$$a_{j,j-1}^{(k)} = O\left(\left|\frac{\lambda_j - \mu}{\lambda_{j-1} - \mu}\right|^k\right), \qquad j = 2, 3, \ldots, n, \qquad k = 1, 2, \ldots$$

$\Rightarrow a_{j,j-1}^{(k)} \xrightarrow{k\to\infty} 0$ fast if $\mu \approx \lambda_j$.

Strategy for Choosing μ_k

At the beginning of the kth iteration of the QR algorithm,

$$A^{(k-1)} = [a_{ij}^{(k-1)}] = \begin{bmatrix} * & \cdots & \cdots & \cdots & * \\ a_{21}^{(k)} & * & & & \vdots \\ 0 & \ddots & \ddots & & 1 \\ \vdots & \ddots & \ddots & \ddots & \vdots \\ 0 & \cdots & 0 & a_{n,n-1}^{(k)} & * \end{bmatrix},$$

$j = 2, 3, \ldots, n,$ check if $a_{j,j-1}^{(k-1)} \approx 0.$

Strategy for Choosing μ_k: Deflation Test

$$\begin{matrix} & & a_{j-1,j-1}^{(k)} \\ & & \\ a_{j,j-1}^{(k-1)} & & a_{jj}^{(k)} \end{matrix}$$

$$\left| a_{j,j-1}^{k-1} \right| \leqslant \epsilon \left(\left| a_{j-1,j-1}^{(k-1)} \right| + \left| a_{jj}^{(k-1)} \right| \right), \qquad \text{where } \epsilon = O(e_{ps})$$

Set $a_{j,j-1}^{(k-1)} = 0$.

Case 1

Case 1: $j = n$

$$A^{(k-1)} = \left[\begin{array}{ccccc|c} * & * & \cdots & \cdots & * & * \\ * & \ddots & \ddots & & \vdots & \vdots \\ & \ddots & \ddots & \ddots & \vdots & \vdots \\ & & \ddots & \ddots & * & \vdots \\ & & & * & * & * \\ \hline 0 & \cdots & \cdots & 0 & 0 & a_{nn}^{(k-1)} \end{array}\right] = \left[\begin{array}{c|c} \tilde{A}^{(k-1)} & * \\ \hline 0 & a_{nn}^{(k-1)} \end{array}\right], \quad \tilde{A}^{(k-1)} \in \mathbb{C}^{n-}$$

$\Rightarrow a_{nn}^{(k)} \in \Lambda(A^{(k-1)} = \Lambda(A)$.

Accept $a_{nn}^{(k)}$ as an eigenvalue of A and continue the QR algorithm on $\tilde{A}^{(k-1)}$.

Typical choice: $\mu_k = a_{n-1,n-1}^{(k-1)}$.

Case 2

Case 2: $j = 2$

$$A^{(k-1)} = \left[\begin{array}{c|ccc} a_{11}^{(k-1)} & * & \cdots & * \\ \hline 0 & & & \\ 0 & & & \\ \vdots & & \tilde{A}^{(k-1)} & \\ 0 & & & \end{array}\right]$$

$a_{11}^{(k-1)}$ is an eigenvalue of A.

Case 3

Case 3: $2 < j < n$

$$A^{(k-1)} = \left[\begin{array}{ccccc|ccccc} * & * & \cdots & \cdots & * & * & \cdots & \cdots & \cdots & * \\ * & \ddots & \ddots & & \vdots & \vdots & & & & \vdots \\ & \ddots & \ddots & \ddots & \vdots & \vdots & & & & \vdots \\ & & \ddots & \ddots & * & \vdots & & & & \vdots \\ & & & * & * & * & \cdots & \cdots & \cdots & * \\ \hline & & & & 0 & * & * & \cdots & \cdots & * \\ & & & & & * & \ddots & \ddots & & \vdots \\ & & 0 & & & & \ddots & \ddots & \ddots & \vdots \\ & & & & & & & \ddots & \ddots & * \\ & & & & & 0 & & & * & * \end{array}\right] = \left[\begin{array}{c|c} A_1^{(k-1)} & * \\ \hline 0 & A_2^{(k-1)} \end{array}\right]$$

Case 3

$$A^{(k-1)} = \left[\begin{array}{c|c} A_1^{(k-1)} & * \\ \hline 0 & A_2^{(k-1)} \end{array}\right]$$

$\Lambda(A) = \Lambda(A^{(k-1)}) = \Lambda(A_1^{(k-1)}) \cup \Lambda(A_2^{(k-1)})$. Continue QR on $A_1^{(k-1)} \in \mathbb{C}^{j-1 \times j-1}$ and $A_2^{(k-1)} \in \mathbb{C}^{n-j+1 \times n-j+1}$.

Flop Count

Typical flop count for such a practical QR algorithm:

- $\sim 10n^3$ if only the eigenvalues are computed
- $\sim 27n^3$ if eigenvalues and eigenvectors are computed